NUMPY ET PANDAS

Le calcul et les données à grande vitesse

NUMPY ET PANDAS

	col_A	col_B	col_C
0	…	…	…
1	…	…	…
2	…	…	…

LES TABLEAUX NUMPY

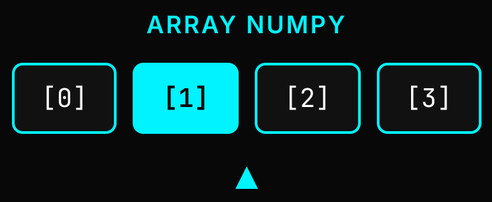
Le tapis roulant ultra-rapide

```python
import numpy as np

# Creer un array depuis une liste
notes = np.array([12, 15, 8, 18, 14])
print(notes.dtype)   # int64
print(notes.shape)   # (5,)

# Operations vectorisees (pas de boucle !)
notes_sur_20 = notes / 20 * 100
print(notes_sur_20)  # [60. 75. 40. 90. 70.]

# Comparaison : 100x plus rapide qu'une boucle
bonnes = notes[notes ≥ 14]
print(bonnes)  # [15. 18. 14.]
```



STATISTIQUES ET GÉNÉRATION

mean, std, arange, linspace

```python
import numpy as np

notes = np.array([12, 15, 8, 18, 14, 11, 16])

# Statistiques en une ligne
print(np.mean(notes))   # 13.43
print(np.std(notes))    # 3.15
print(np.sum(notes))    # 94
print(np.min(notes), np.max(notes))  # 8 18

# Generer des sequences
x = np.arange(0, 10, 0.5)      # 0, 0.5, 1, ... 9.5
y = np.linspace(0, 1, 5)       # 0, 0.25, 0.5, 0.75, 1.0
```

NP.MEAN() NP.STD()

`np.mean() np.std()`

SERIES ET DATAFRAME

Le tableau Excel surpuissant

```python
import pandas as pd

# Creer un DataFrame depuis un dictionnaire
df = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'Clara', 'David'],
    'note': [18, 12, 15, 9],
    'ville': ['Paris', 'Lyon', 'Paris', 'Lille']
})
print(df)
#      nom  note  ville
# 0  Alice    18  Paris
# 1    Bob    12   Lyon
# 2  Clara    15  Paris
```

DATAFRAME PANDAS

	col_A	col_B	col_C
0	…	…	…
1	…	…	…
2	…	…	…

CHARGER DES DONNÉES

read_csv, head, info, describe

```python
import pandas as pd

# Charger un CSV en une ligne
df = pd.read_csv('ventes.csv')

# Explorer les donnees
print(df.head())       # 5 premieres lignes
print(df.shape)        # (1000, 5) → 1000 lignes, 5 colo
print(df.info())       # types et valeurs manquantes
print(df.describe())   # stats : count, mean, std, min,

# Autres formats supportes
# pd.read_excel('fichier.xlsx')
```

PD.READ_CSV()

📄 fichier ↔ 🐍 Python

SÉLECTION AVEC LOC ET ILOC

Accéder aux lignes et colonnes

```python
import pandas as pd

df = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'Clara'],
    'note': [18, 12, 15],
    'ville': ['Paris', 'Lyon', 'Paris']
})

# loc : acces par etiquette
print(df.loc[0, 'nom'])        # 'Alice'
print(df.loc[0:1, ['nom', 'note']])

# iloc : acces par position
print(df.iloc[0, 0])           # 'Alice'
```

LOC[] ET ILOC[]

[0] [1] [2] [3]

FILTRER LES DONNÉES

Conditions et masques booléens

```python
import pandas as pd

df = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'Clara', 'David'],
    'note': [18, 12, 15, 9],
    'ville': ['Paris', 'Lyon', 'Paris', 'Lille']
})

# Filtre simple
bons = df[df['note'] ≥ 14]
print(bons)  # Alice, Clara

# Conditions combinees (& = et, | = ou)
```

FILTRAGE BOOLÉEN

données ▶ Filtrage booléen ▶ résultat

TRIER LES DONNÉES

sort_values et sort_index

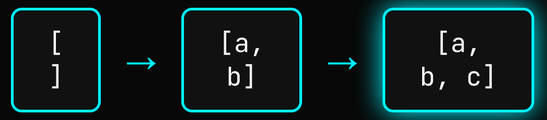
```python
import pandas as pd

df = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'Clara', 'David'],
    'note': [18, 12, 15, 9],
    'ville': ['Paris', 'Lyon', 'Paris', 'Lille']
})

# Tri par une colonne (croissant par defaut)
print(df.sort_values('note'))

# Tri decroissant
print(df.sort_values('note', ascending=False))
```

TRI CROISSANT/DÉCROISSANT



COLONNES CALCULÉES

assign et apply

```python
import pandas as pd

df = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'Clara'],
    'note': [18, 12, 15]
})

# Nouvelle colonne par calcul direct
df['note_100'] = df['note'] / 20 * 100

# assign retourne une copie
df2 = df.assign(mention=lambda x: x['note'] ≥ 14)

# apply pour une logique personnalisée
```

NOUVELLES COLONNES

~~Nouvelles colonnes~~ → après

FUSIONNER DES DATAFRAMES

merge et concat

```python
import pandas as pd

# Deux tables a fusionner
df_notes = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'Clara'],
    'note': [18, 12, 15]
})
df_villes = pd.DataFrame({
    'nom': ['Alice', 'Bob', 'David'],
    'ville': ['Paris', 'Lyon', 'Lille']
})

# merge : jointure sur la colonne 'nom'
```

JOINDRE DEUX TABLES

Joindre deux tables **VS** Joindre deux tables

GROUPER ET AGRÉGER

groupby et agrégation

```python
import pandas as pd

df = pd.DataFrame({
    'ville': ['Paris', 'Lyon', 'Paris', 'Lyon', 'Paris'],
    'ventes': [100, 80, 120, 90, 110],
    'cout': [50, 40, 60, 45, 55]
})

# Grouper par ville, calculer la moyenne
print(df.groupby('ville')['ventes'].mean())
# Lyon     85.0
# Paris   110.0

# Plusieurs agrégations à la fois
```

GROUPBY → AGG → RÉSULTAT

RÉCAPITULATIF

NumPy et pandas

- ✓ NumPy : arrays rapides et opérations vectorisées
- ✓ Statistiques : mean, std, sum, min, max
- ✓ DataFrame : tableau structuré avec index
- ✓ read_csv, head, info, describe pour explorer
- ✓ loc/iloc pour sélectionner, filtres booléens
- ✓ Tri avec sort_values, fusion avec merge et concat
- ✓ assign/apply pour créer des colonnes
- ✓ groupby + agg pour agréger par catégorie